# Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #9

Side effects

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Agenda

- **Built in predicates**
  - **Side Effects:** `assert` **and** `retract`
- Utilization on complex tasks
  - Towers of Hanoi

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Side effects

- Allow for persistent structure
- Allow for wider scope visibility (modelling global variables; created in some predicate and visible elsewhere, without passing arguments)
- Allow for metaprogramming
- By using them, we exceed the pure Logic Programming (operations NOT undone on backtracking)

# Side effects

- Built in predicates
- **assert/retract** for adding/extracting ONE clause of the predicate with the name provided as argument
- The argument (for both assert and retract) is a variable representing a dynamic predicate
  - `assert(X)` / `retract(X)`
- The variable *X* must be ALREADY instantiated for the predicate to be updated

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Side effects – assert

- assert comes into 2 flavors:
  - **`asserta`** – adds in the beginning (on top) of the existing clauses for the predicate provided as argument
  - **`assertz / assert`** – adds at the end (at bottom) of the existing clauses for the predicate provided as argument
- It NEVER fails
- on backtracking, the item (clause) is NOT removed. However, assert does not re-execute on backtracking (NO new clause is added).

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Side effects – retract

- removes the first item (clause of the predicate provided as argument) with whom it matches successfully
- if no match (the matching fails), retract fails. So, as opposed to assert, it can fail
- on backtracking, the item (clause removed) is NOT added back
  - On the contrary, another item is removed in case another successful match is possible, ***or*** retract fails

# assert

- Assume we have some (3 in the ex.) clauses for a predicate `p(…)` (arguments do not matter here).
- Here on the right, the clauses have numbers added to represent their index

```
p(…) :-…        p1(…) :-…
p(…) :-…        p2(…) :-…
p(…) :-…        p3(…) :-…
```

# asserta/assertz

- After the execution of `asserta(p(…))`, the predicate becomes:

$p_{new}$(…) :-…
$p_1$(…) :-…
$p_2$(…) :-…
$p_3$(…) :-…

- After the execution of `assertz(p(…))`, the predicate becomes:

$p_1$(…) :-…
$p_2$(…) :-…
$p_3$(…) :-…
$p_{new}$(…) :-…

# Assert example

```
:-dynamic p/1.

p(1).
p(2).
p(3).
p(4).
p(5).

q:- assertz(p(6)), fail.
q.

?- q.
```

error (No permission to modify static procedure `p/1')

true.

No result is actually shown, it just results in a true.

```
?- listing(p/1).
```

p(1).
p(2).
p(3).
p(4).
p(5).

```
?- q, listing(p/1).
```

p(1).
p(2).
p(3).
p(4).
p(5).
p(6).

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# retract

- Assume this scenario:

```
p1(…) :-…
p2(…) :-…
p3(…) :-…
```

- The execution of `retract(p())` would:
  - Start scanning the list of clauses of the predicate **`p`**
  - Stop at the first clause where arguments (not mentioned here) match
  - Remove the given clause and report success
  - If no successful matching, the retract fails

# retract – success on the first clause

- Assume the same scenario:

```
p1(…):-…
p2(…):-…
p3(…):-…
```

- If execution of `retract(p())` succeeds on matching with **first** clause of `p`, it leads to:

```
p2(…):-…
p3(…):-…
```

# retract – success on the second clause

- Assume the same scenario:

```
p1(…):-…
p2(…):-…
p3(…):-…
```

- If execution of `retract(p())` succeeds on matching with **second** clause of `p`, it leads to:

```
p1(…):-…
p3(…):-…
```

# retract – success on the third clause

- Assume the same scenario:

```
p1(…):-…
p2(…):-…
p3(…):-…
```

- If execution of `retract(p())` succeeds on matching with **third** clause of `p`, it leads to:

```
p1(…):-…
p2(…):-…
```

# retract – fails

- Assume the same scenario:

```
p1(…) :-…
p2(…) :-…
p3(…) :-…
```

- If execution of `retract(p())` does NOT succeed trying to match `p` to ANY of the 3 clauses, the result would be:

```
p1(…) :-…
p2(…) :-…
p3(…) :-…
```

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# retract – on backtracking

- After the execution of retract, on backtracking, NO undo is performed (the removal stays). That clause is NEVER restored, unless an explicit request (assert) is made
- Moreover, on the attempt to REsatisfy the goal (due to backtracking), Prolog continues FROM THE PLACE it previously removed and tries to remove ANOTHER clause, and:
  - it does so if the match succeeds,
  - or else fails and the execution is resumed (backtracked) to the predicate on the left of retract

# retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

```
p1(…):-…
p2(…):-…
p3(…):-…
p4(…):-…
p5(…):-…
```

- Assume the execution of `retract(p(_))` succeeds on matching with the **second** clause of predicate `p`, after execution the predicate contains:

```
p1(…):-…
p3(…):-…
p4(…):-…
p5(…):-…
```

# retract – on backtracking - contd

- If now we backtrack again, and retract matches the 3rd clause (i.e. `p3` which is the second actually), the predicate looks:

```
p1(…):-…
p4(…):-…
p5(…):-…
```

- If we backtrack again, and retract matches the 5th clause (i.e. `p5` which is the third actually) the predicate looks:

```
p1(…):-…
p4(…):-…
```

- A new backtrack now would fail, as we reached the end of the list of clauses.

# Retract example

```
:-dynamic p/1.

p(1).
p(2).
p(3).
p(4).
p(5).

q:- retract(p(_)), fail.
q.

?- q.
```

error (No permission to modify static procedure `p/1‘)

true.

No result is actually shown, it just results in a true.

```
?- listing(p/1).
p(1).
p(2).
p(3).
p(4).
p(5).

?- q, listing(p/1).
```

true.

# retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1$(…) :-…
$p_2$(…) :-…
$p_3$(…) :-…
$p_4$(…) :-…
$p_5$(…) :-…

- Assume the execution of `retract(p(_))` cannot unify with ANY of the 5 clauses, the execution fails and `p` remains unchanged:

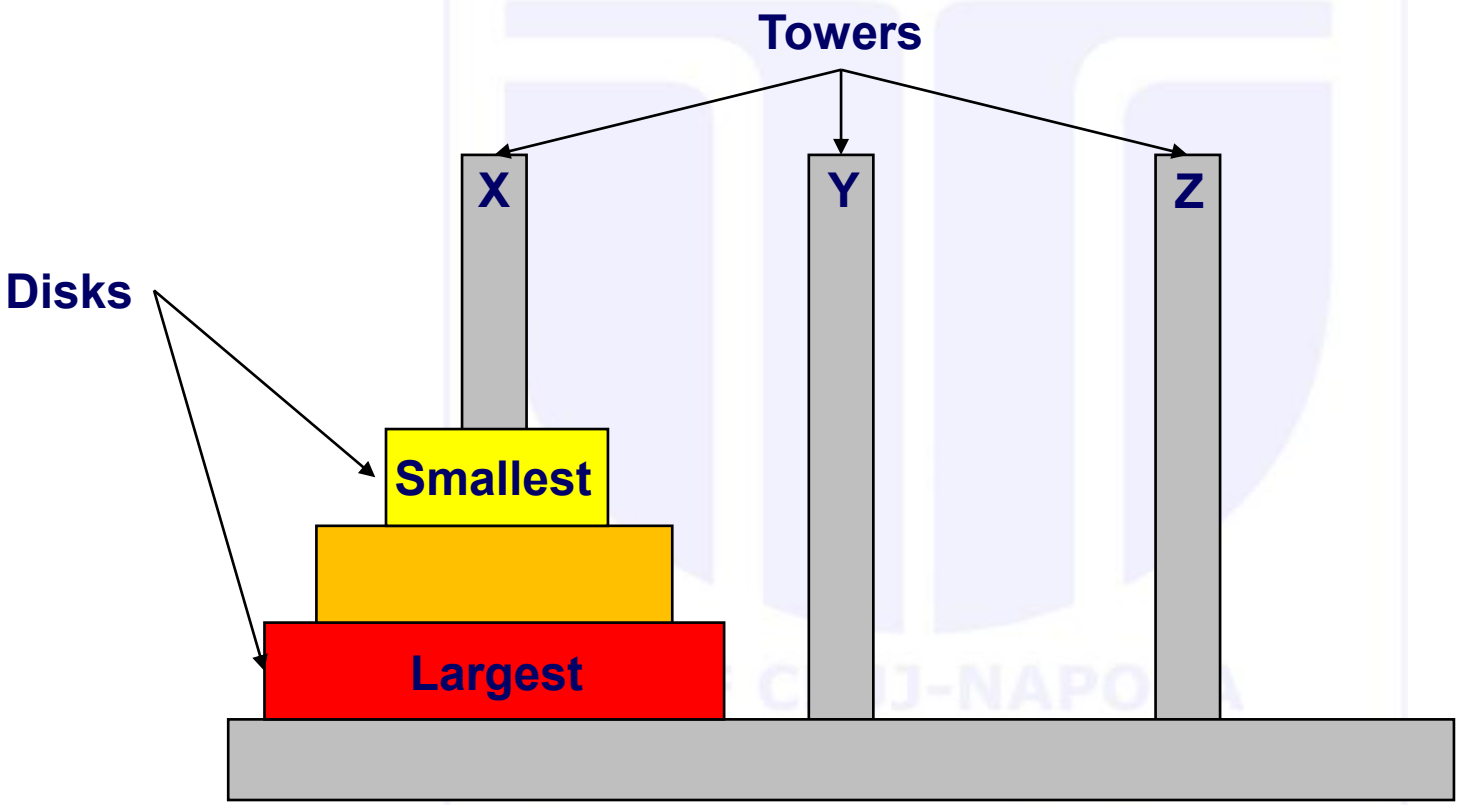
$p_1$(…) :-…
$p_2$(…) :-…
$p_3$(…) :-…
$p_4$(…) :-…
$p_5$(…) :-…

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# assert and retract

- Combining assert and retract
  - `asserta` + `retract` => LIFO
  - `assertz` + `retract` => FIFO

# Towers of Hanoi – the problem



TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Towers of Hanoi - formulation

- Given *n* disks and *3* towers/poles
- Move the disks from tower *X* to *Z* (same order – smallest to largest)
- At any time, the constraints are:
  - One disk at a time
  - Just the top disk is accessible
  - On any tower, any disk has the diameter smaller than the diameter of the disk on top of which it stays

# Towers of Hanoi - rephrasing

Start with *n* disks on *X* and end with *n* disks on *Z*

$$X \xrightarrow{(n,Y)} Z \quad \text{means} \quad X \xrightarrow{(n-1,Z)} Y \;\; X \to Z \;\; Y \xrightarrow{(n-1,X)} Z$$

- Means:
  - Move n discs from X to Z using Y as intermediate by means of:
    - Move (n-1) discs from X to Y using Z as intermediate
    - Move one single disc from X to Z
    - Move (n-1) discs from Y to Z using X as intermediate

# Towers of Hanoi - rephrasing

Start with *n* disks on *X* and end with *n* disks on *Z*

$X \xrightarrow{(n,Y)} Z$ means $X \xrightarrow{(n-1,Z)} Y$, $X \rightarrow Z$, $Y \xrightarrow{(n-1,X)} Z$

BUT this is almost Prolog code:

```
move_discs(N,X,Y,Z):-
        N1 is N-1,
        move_discs(N1,X,Z,Y),
        move_disc(X,Z),
        move_discs(N1,Y,X,Z).
```

# Towers of Hanoi - realization

- Disc representation (as stack of facts in the knowledge base):

```
x(1).
⋮
x(n-1).
x(n).          % n is NOT a variable, is a value.
```

- How is it realized? `% init_tower/2 (+input, +List)`

```
init_tower(_,[]):-!.
init_tower(T,[N|L]):-
      D=..[T,N],          % T name of tower, as x in example; N size
      asserta(D),         % put on top
      init_tower(T,L).
```

- Where `L` is a list of integers (diameters of discs) in decreasing order

```
init_list(1,[1]):-!.
init_list(N,[N|Rest]):-
      N1 is N-1,
      init_list(N1,Rest).
```

# Towers of Hanoi – moving one disk

- Move one disc from Input Pole to Output Pole:

```
move_disc(IP,OP):-
        DI=..[IP,Size],     % IP name of input pole, Size diam of disk
        retract(DI),!,      % extract from top
        DO=..[OP,Size],     % IP name of output pole, Size diam of disk
        asserta(DO).        % put on top
```

- If input has:

```
x(3).
x(4).          y(6).
x(5).          y(7).
```

- And call `move_disc(x,y).` goes to:

```
               y(3).
x(4).          y(6).
x(5).          y(7).
```

# Towers of Hanoi – complete code

```
move_discs(0,_,_,_):-!.
move_discs(N,X,Y,Z):-
        N1 is N-1,
        move_discs(N1,X,Z,Y),
        move_disc(X, Z),
        move_discs(N1,Y,X,Z).

move_disc(IP,OP):-
        DI=..[IP,Size],
        retract(DI),!,
        DO=..[OP,Size],
        asserta(DO).

init_tower(_,[]):-!.
init_tower(P,[N|L]):-
        D=..[P,N],
        asserta(D),
        init_tower(P,L).

init_list(1,[1]):-!.
init_list(N,[N|Rest]):-
        N1 is N-1,
        init_list(N1,Rest).
```

```
% INITIAL call
hanoi(N):-
        init_list(N,List),
        init_tower(in,List),
        move_discs(N,in,int,out).
```

```
[q] ?- retractall(out(_)),
hanoi(3), listing(in/1),
listing(out/1).
```